

uni-app scroll-view（可滚动视图区域） 组件详解（适配 uni-app 2.2+）

uni-app scroll-view（可滚动视图区域）组件详解（适配 uni-app 2.2+）

`scroll-view` 是 uni-app 核心内置组件，用于实现**局部滚动**（区别于页面全局滚动），支持纵向/横向滚动、下拉刷新、上拉加载、滚动监听等核心能力，在 2.2+ 版本中已完善大部分核心功能，以下是全维度使用指南：

一、核心特性与适用场景

1. 核心能力：

- 支持纵向（默认）/横向滚动；
- 可自定义滚动位置、滚动到指定节点；
- 支持下拉刷新、上拉加载触底；
- 滚动事件监听（滚动中、滚动到底、滚动到顶等）；
- 支持滚动惯性、回弹效果（可配置）。

2. 适用场景：

- 长列表局部滚动（如页面内的商品列表、消息列表）；
- 横向滚动导航栏、横向卡片列表；
- 自定义下拉刷新/上拉加载（替代页面级的 onPullDownRefresh/onReachBottom）；
- 需精准控制滚动位置的场景（如滚动到指定锚点）。

二、基础属性（uni-app 2.2+ 核心属性）

属性名	类型	默认值	说明
<code>scroll-x</code>	Boolean	false	是否允许横向滚动
<code>scroll-y</code>	Boolean	false	是否允许纵向滚动 必须给 <code>scroll-view</code> 高度
<code>upper-threshold</code>	Number	50	距顶部/左边多远时（单位 px），触发 <code>scrolltoupper</code> 事件

<code>lower-threshold</code>	Number	50	距底部/右边多远时(px)，触发 <code>scrolltolower</code>
<code>scroll-top</code>	Number	-	设置竖向滚动条位置，改需用 <code>:scroll-top</code> （固定）
<code>scroll-left</code>	Number	-	设置横向滚动条位置，改需用 <code>:scroll-left</code> （固定）
<code>scroll-into-view</code>	String	-	值应为子元素 id（以字母开头），滚动到指定元素
<code>scroll-with-animation</code>	Boolean	false	滚动时是否使用动画
<code>enable-back-to-top</code>	Boolean	false	iOS 点击顶部状态栏时，是否回到顶部（仅支持竖向）
<code>enable-flex</code>	Boolean	false	是否启用 flex 布局，改需用 <code>display: flex</code>
<code>refresher-enabled</code>	Boolean	false	是否开启自定义下拉刷新
<code>refresher-triggered</code>	Boolean	false	下拉刷新状态（true 表示正在刷新中，需手动控制）
<code>refresher-threshold</code>	Number	45	下拉刷新触发阈值(px)
<code>refresher-default-style</code>	String	"black"	下拉刷新默认样式（black/white/none）
<code>refresher-background</code>	String	"#ffffff"	下拉刷新区域背景色
<code>bounce</code>	Boolean	true	是否启用回弹效果（滚动到边界无弹性）
<code>show-scrollbar</code>	Boolean	true	是否显示滚动条（小程序支持，H5 暂不支持）

三、核心事件（uni-app 2.2+ 支持）

事件名	说明	回调参数
-----	----	------

scrolltoupper	滚动到顶部/左边时触发	{ scrollTop, scrollLeft, isDragging }
scrolltolower	滚动到底部/右边时触发	{ scrollTop, scrollLeft, isDragging }
scroll	滚动过程中触发（频繁触发，需注意性能）	{ scrollTop, scrollLeft, deltaX, deltaY }
refresherpulling	下拉刷新拖动过程中触发	-
refresherrefresh	下拉刷新触发时触发	-
refresherrestore	下拉刷新复位时触发	-
refresherabort	下拉刷新终止时触发	-

四、经典使用示例

示例1：纵向滚动列表（基础款）

Code block

```
1  <template>
2    <!-- 纵向滚动必须给 scroll-view 固定高度 -->
3    <scroll-view
4      class="scroll-y-container"
5      scroll-y
6      @scrolltolower="loadMore"
7      @scrolltoupper="toTop"
8      :scroll-top="scrollTop"
9      lower-threshold="100"
10   >
11    <!-- 列表内容 -->
12    <view class="item" v-for="(item, index) in list" :key="index">
13      列表项 {{ index + 1 }}: {{ item }}
14    </view>
15    <!-- 加载中提示 -->
16    <view class="load-more" v-if="loading">加载中...</view>
17  </scroll-view>
18 </template>
19
20 <script setup>
21 import { ref, onLoad } from "vue";
22
23 // 响应式数据
24 const list = ref([]);
```

```
25  const loading = ref(false);
26  const scrollTop = ref(0); // 控制滚动条位置
27
28  // 初始化数据
29  onLoad(() => {
30    loadData();
31  });
32
33  // 加载列表数据
34  const loadData = () => {
35    for (let i = 0; i < 20; i++) {
36      list.value.push(`内容${i + 1}`);
37    }
38  };
39
40  // 上拉加载更多
41  const loadMore = () => {
42    if (loading.value) return;
43    loading.value = true;
44    // 模拟接口请求
45    setTimeout(() => {
46      loadData();
47      loading.value = false;
48    }, 1000);
49  };
50
51  // 滚动到顶部触发
52  const toTop = () => {
53    uni.showToast({ title: "已到顶部", icon: "none" });
54  };
55  </script>
56
57  <style scoped>
58  /* 纵向滚动容器必须设置高度 */
59  .scroll-y-container {
60    height: 600rpx; /* 适配小程序/APP/H5 */
61    width: 100%;
62    background: #f5f5f5;
63  }
64  .item {
65    height: 80rpx;
66    line-height: 80rpx;
67    padding: 0 20rpx;
68    border-bottom: 1px solid #eee;
69    background: #fff;
70  }
71  .load-more {
```

```
72     text-align: center;
73     padding: 20rpx 0;
74     color: #999;
75 }
76 </style>
```

示例2：横向滚动导航栏

Code block

```
1  <template>
2    <scroll-view
3      class="scroll-x-container"
4      scroll-x
5      enable-flex
6      show-scrollbar="false"
7    >
8      <view class="nav-item" v-for="(item, index) in navList" :key="index">
9        {{ item }}
10     </view>
11   </scroll-view>
12 </template>
13
14 <script setup>
15 const navList = ref(["首页", "推荐", "热点", "视频", "直播", "商城", "我的"]);
16 </script>
17
18 <style scoped>
19 .scroll-x-container {
20   white-space: nowrap; /* 关键：防止子元素换行 */
21   background: #fff;
22   border-bottom: 1px solid #eee;
23 }
24 .nav-item {
25   display: inline-block; /* 横向排列核心 */
26   padding: 0 30rpx;
27   height: 88rpx;
28   line-height: 88rpx;
29   font-size: 28rpx;
30 }
31 </style>
```

示例3：自定义下拉刷新

Code block

```
1  <template>
2    <scroll-view
3      class="scroll-y-container"
4      scroll-y
5      refresher-enabled
6      :refresher-triggered="refreshing"
7      @refresherrefresh="onRefresh"
8      @refresherpulling="onPulling"
9      refresher-background="#f5f5f5"
10   >
11     <view class="item" v-for="(item, index) in list" :key="index">
12       刷新后的列表 {{ index + 1 }}
13     </view>
14   </scroll-view>
15 </template>
16
17 <script setup>
18 import { ref } from "vue";
19
20 const list = ref([]);
21 const refreshing = ref(false); // 刷新状态
22
23 // 初始化数据
24 const initData = () => {
25   list.value = [];
26   for (let i = 0; i < 15; i++) {
27     list.value.push(`初始内容${i + 1}`);
28   }
29 };
30 initData();
31
32 // 下拉刷新触发
33 const onRefresh = () => {
34   refreshing.value = true;
35   // 模拟接口刷新数据
36   setTimeout(() => {
37     list.value = [];
38     for (let i = 0; i < 15; i++) {
39       list.value.push(`刷新后内容${i + 1}`);
40     }
41     refreshing.value = false; // 结束刷新
42   }, 1500);
43 };
44
45 // 下拉拖动过程中 (可选)
46 const onPulling = (e) => {
47   console.log("下拉距离: ", e.detail.dy);
```

```
48  };
49  </script>
50
51  <style scoped>
52  .scroll-y-container {
53    height: 700rpx;
54    width: 100%;
55    background: #f5f5f5;
56  }
57  .item {
58    height: 80rpx;
59    line-height: 80rpx;
60    padding: 0 20rpx;
61    background: #fff;
62    border-bottom: 1px solid #eee;
63  }
64  </style>
```

示例4：滚动到指定元素（scroll-into-view）

Code block

```
1  <template>
2    <view>
3      <button @click="scrollToTarget">滚动到第10项</button>
4      <scroll-view
5        class="scroll-y-container"
6        scroll-y
7        :scroll-into-view="targetId"
8        scroll-with-animation
9      >
10     <view
11       class="item"
12       v-for="(item, index) in 20"
13       :key="index"
14       :id="`item-${index}`"
15     >
16       列表项 {{ index + 1 }}
17     </view>
18   </scroll-view>
19 </view>
20 </template>
21
22 <script setup>
23 import { ref } from "vue";
24
```

```
25  const targetId = ref("");
26
27  // 滚动到指定元素
28  const scrollToTarget = () => {
29    targetId.value = "item-9"; // 第10项的id (index从0开始)
30  };
31  </script>
32
33  <style scoped>
34    .scroll-y-container {
35      height: 600rpx;
36      width: 100%;
37      background: #f5f5f5;
38      margin-top: 20rpx;
39    }
40    .item {
41      height: 80rpx;
42      line-height: 80rpx;
43      padding: 0 20rpx;
44      background: #fff;
45      border-bottom: 1px solid #eee;
46    }
47  </style>
```

五、uni-app 2.2+ 版本注意事项

1. 纵向滚动必设高度：

`scroll-y="true"` 时，必须给 `scroll-view` 设置固定高度（如 `height: 600rpx / 100vh`），否则无法滚动（2.2 版本未修复此核心规则）。

2. scroll-into-view 限制：

目标元素 id 不能以数字开头（如 `id="123"` 无效，需改为 `id="item-123"`），且需确保元素已渲染完成（异步数据需加 `nextTick`）。

3. 性能优化：

- `scroll` 事件频繁触发，避免在回调中做复杂计算（可加防抖）；
- 长列表建议结合 `uni-app` 的 `recycle-list` 或虚拟列表（2.2 版本可兼容第三方虚拟列表组件）；
- 关闭不必要的滚动条：`show-scrollbar="false"`（减少渲染开销）。

4. 跨端兼容：

- H5 端：`scroll-view` 的滚动条样式可通过 CSS 自定义（`::-webkit-scrollbar`）；

- 小程序端：`refresher-default-style` 仅支持 black/white，自定义样式需隐藏默认样式后自行实现；
- App 端：回弹效果（`bounce`）默认开启，关闭后滚动到边界无弹性。

5. 常见坑点：

- 横向滚动时，子元素需设置 `display: inline-block` 或 `flex`，且容器不能设 `overflow: hidden`；
- `scroll-top` / `scroll-left` 动态修改时，需用 `v-bind` 绑定，且修改后需确保值为数字；
- 自定义下拉刷新（`refresher-enabled`）与页面级下拉刷新（`onPullDownRefresh`）不可同时使用。

六、进阶技巧

1. 滚动防抖：

对 `scroll` 事件做防抖处理，减少性能消耗：

Code block

```
1 import { ref } from "vue";
2 let timer = null;
3 const onScroll = (e) => {
4   clearTimeout(timer);
5   timer = setTimeout(() => {
6     console.log("防抖后的滚动位置：", e.detail.scrollTop);
7   }, 50);
8 };
```

2. 滚动到底部加载更多防抖：

避免多次触发 `scrolltolower`：

Code block

```
1 const loading = ref(false);
2 const loadMore = () => {
3   if (loading.value) return;
4   loading.value = true;
5   // 加载逻辑...
6   setTimeout(() => {
7     loading.value = false;
8   }, 1000);
9 };
```

3. 自定义滚动条样式（H5 端）：

Code block

```
1  ::v-deep .scroll-y-container::-webkit-scrollbar {  
2    width: 4rpx; /* 滚动条宽度 */  
3  }  
4  ::v-deep .scroll-y-container::-webkit-scrollbar-thumb {  
5    background: #ccc; /* 滚动条滑块颜色 */  
6    border-radius: 2rpx;  
7  }
```

以上是 uni-app 2.2+ 版本 `scroll-view` 组件的完整使用指南，覆盖基础用法、核心场景、避坑点和进阶技巧，可直接适配多端（H5/小程序/App）开发。

（注：文档部分内容可能由 AI 生成）